Data structures

BALANCED TREES

Today's topics

AVL trees

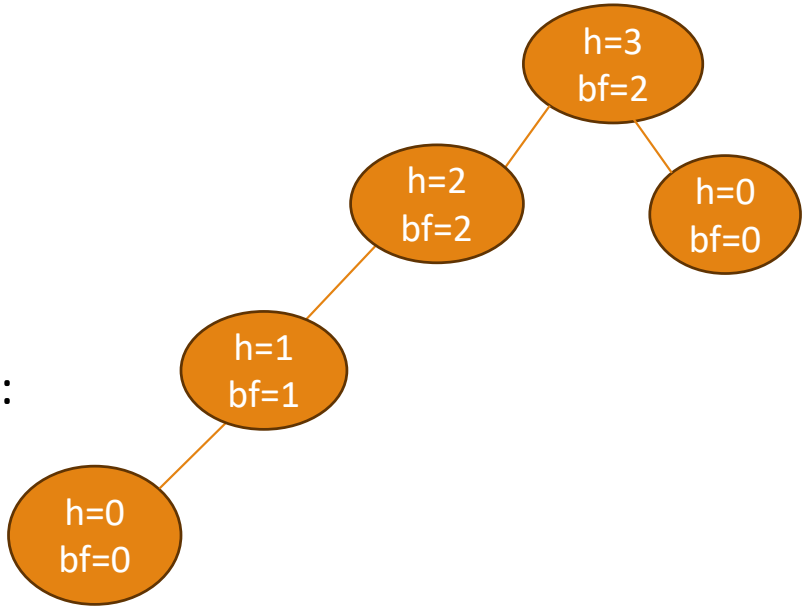
Balanced tree motivation

- We have seen that in a binary search tree, all standard operations have time complexity $O(h)$, where h is the height of the tree.
- In an arbitrary tree, the height is $\mathrm{O}(n)$.
- **Our goal:** a binary search tree whose height is $O(\log(n))$

What is an AVL tree?

- **AVL Trees are:**
  - **Binary Search Trees.**
  - Hold a **height field** for each node.
  - At every node x; **-1 ≤ bf(x) ≤ 1**

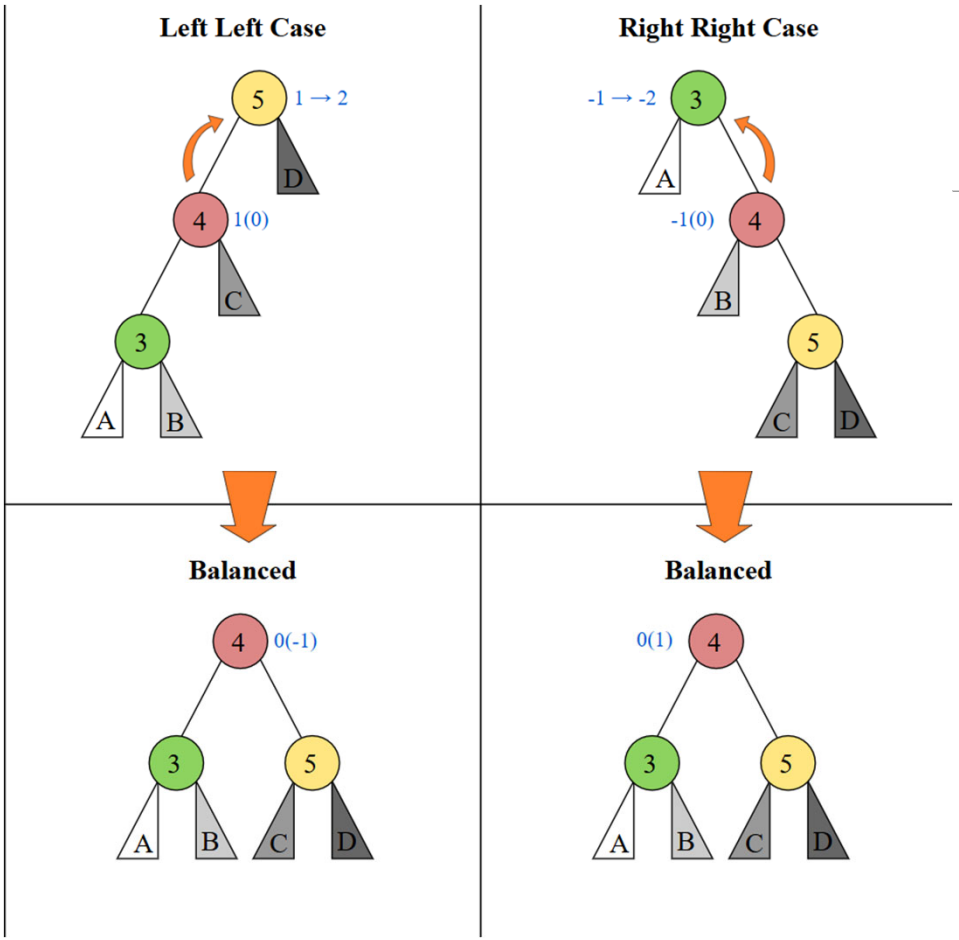
- The **Balance Factor** of node x is defined by:
  bf(x) = height(x.left) - height(x.right)



- An n-nodes AVL tree is always of height O(log n).

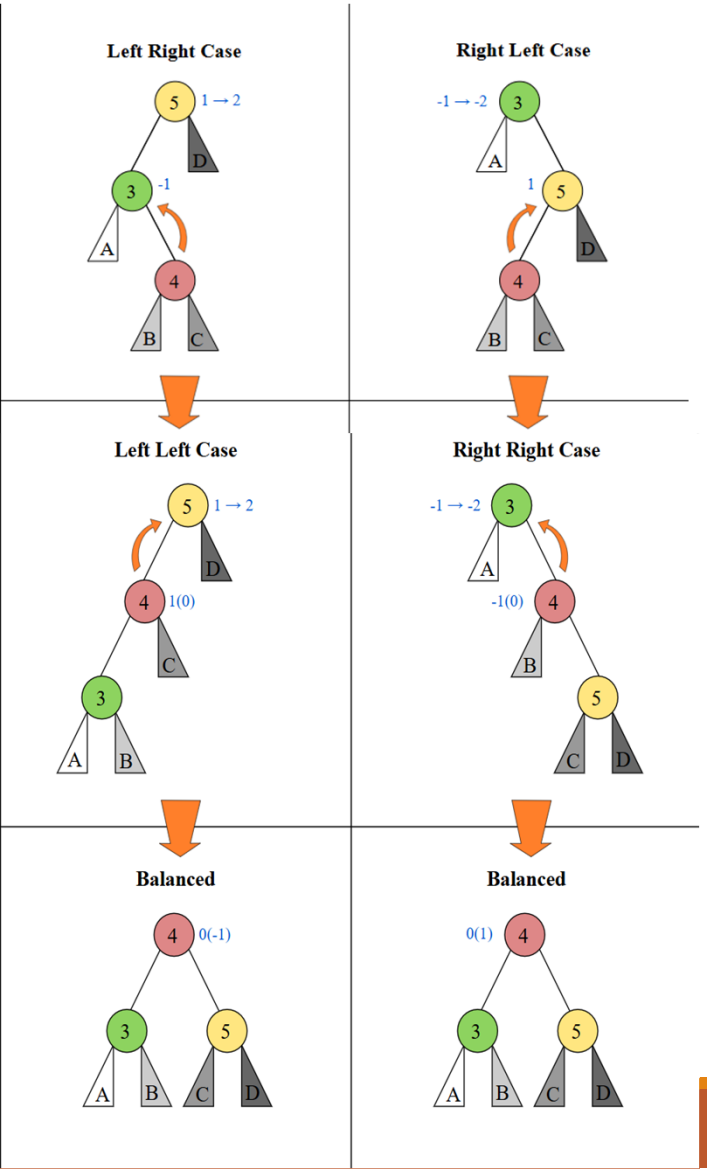
Insertion

- Insertion is done as in the standard binary search tree.
  A new node is inserted as a leaf.
- Adding the leaf might violate the tree's balance property.
- The solution is to fix any imbalances encountered along the path from the new leaf to the root.



Rebalancing

- Let x be the lowest "violating" node.
- Let y be x's heavier son.
- **LL Violation** – both x and y are "left heavy" ($bf \geq 0$).
  Rebalance – Rotate Right (x)
- **RR Violation** – both x and y are "right heavy " ($bf \leq 0$).
  Rebalance – Rotate Left (x)

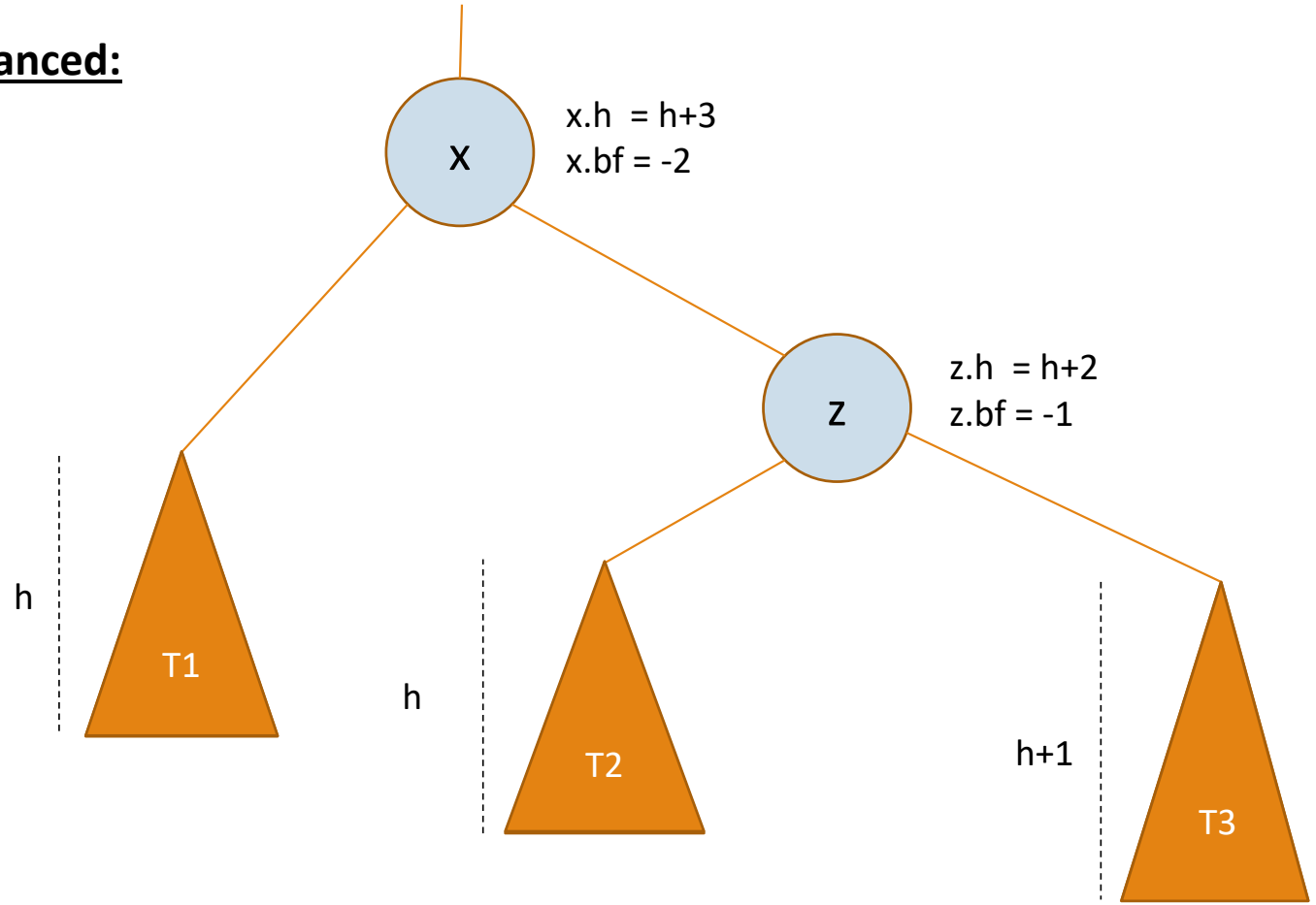


Rebalancing

- Let x be the lowest “violating” node.
- Let y be x’s heavier son.
- **LR Violation** – x is strictly “left heavy” ($bf > 0$).
  and y is strictly “right heavy ” ($bf < 0$).
  Rebalance – Rotate Left (y), Rotate Right (x)
- **RL Violation** – x is strictly “right heavy” ($bf < 0$),
  and y is strictly “left heavy ” ($bf > 0$).
  Rebalance – Rotate Right (y), Rotate Left (x)

AVL Balancing Example - Left Rotation

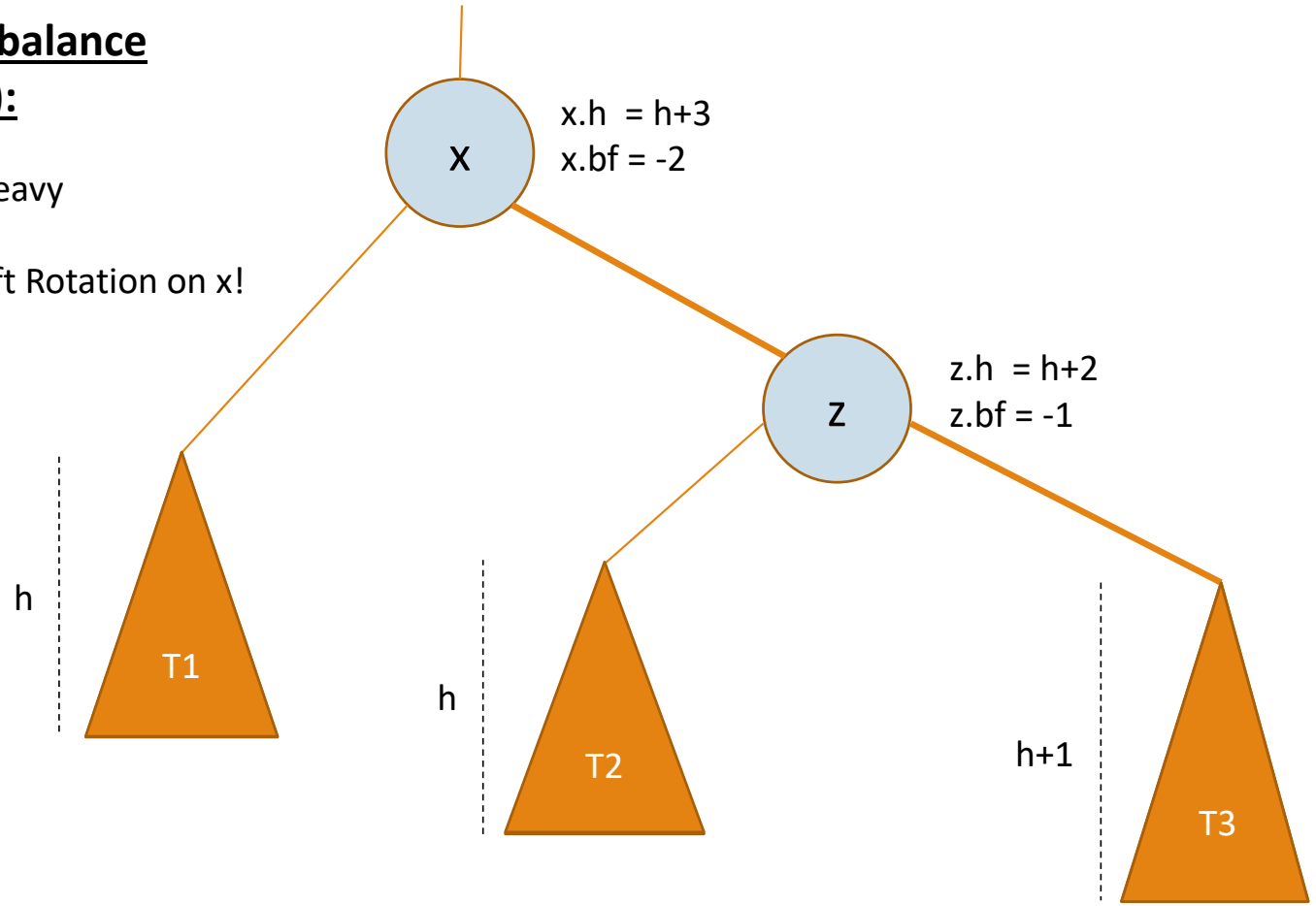
x is unbalanced:



Same Side Imbalance (RR Violation):

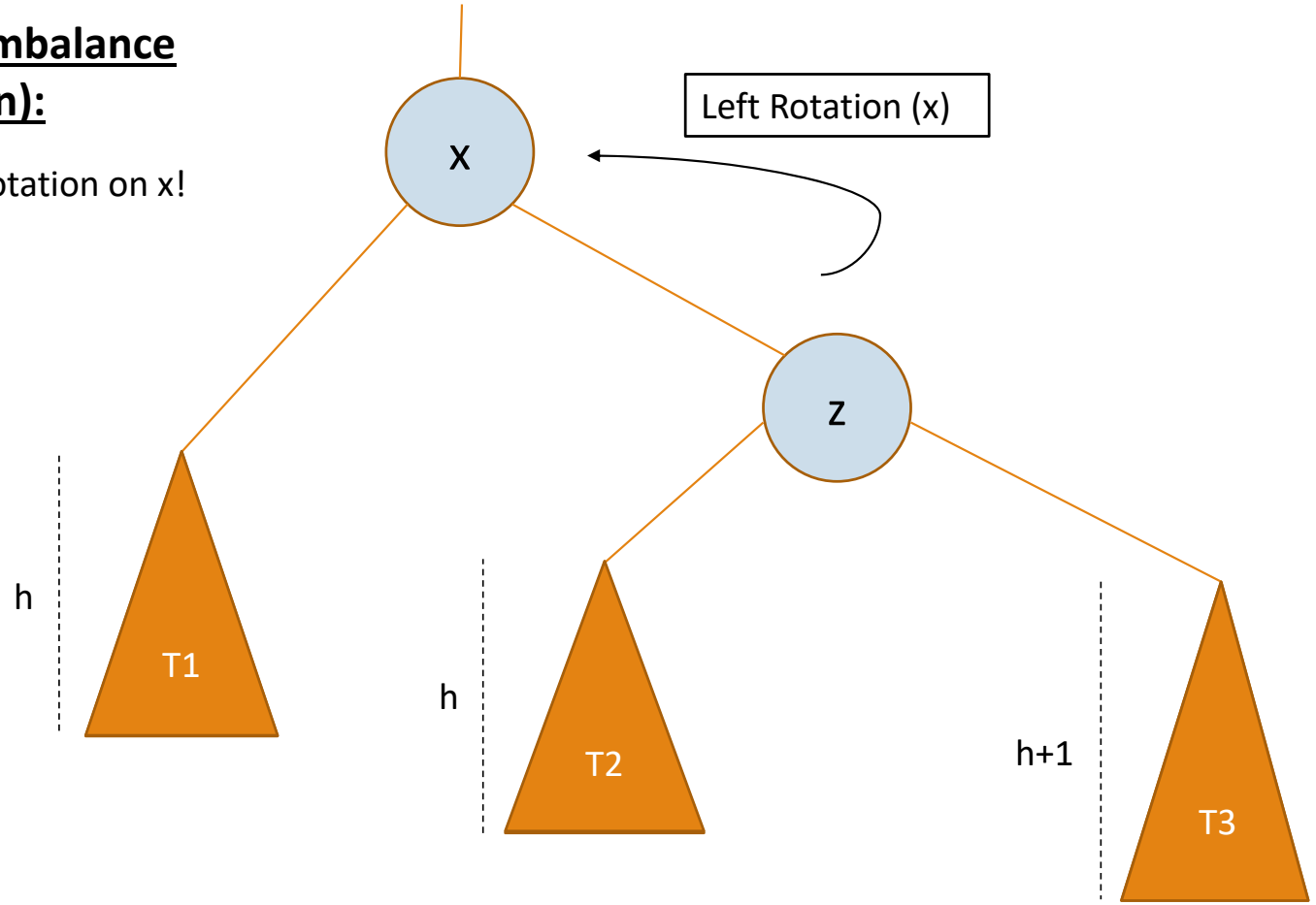
x is right-right heavy

→ Perform a Left Rotation on x!

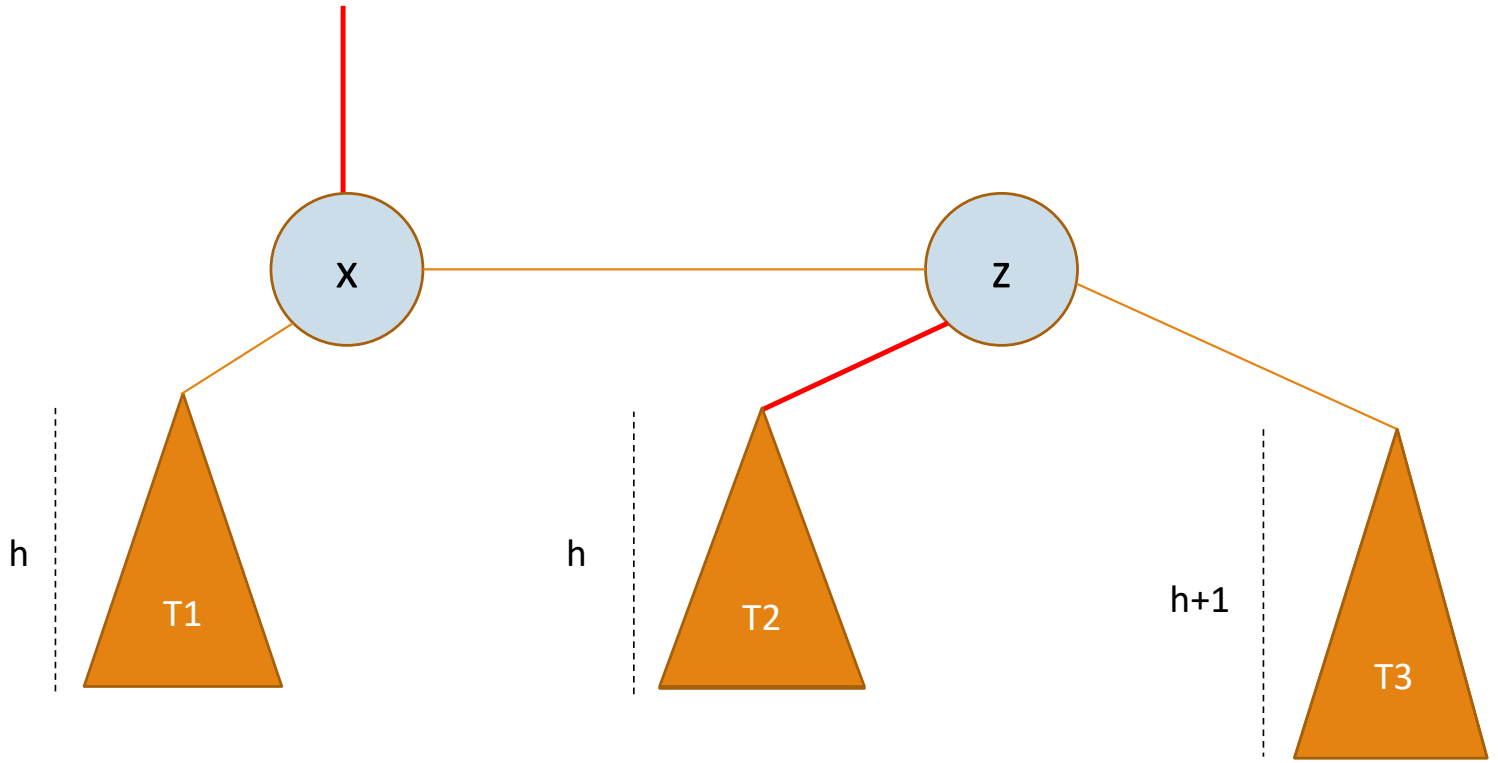


Same Side Imbalance (RR Violation):

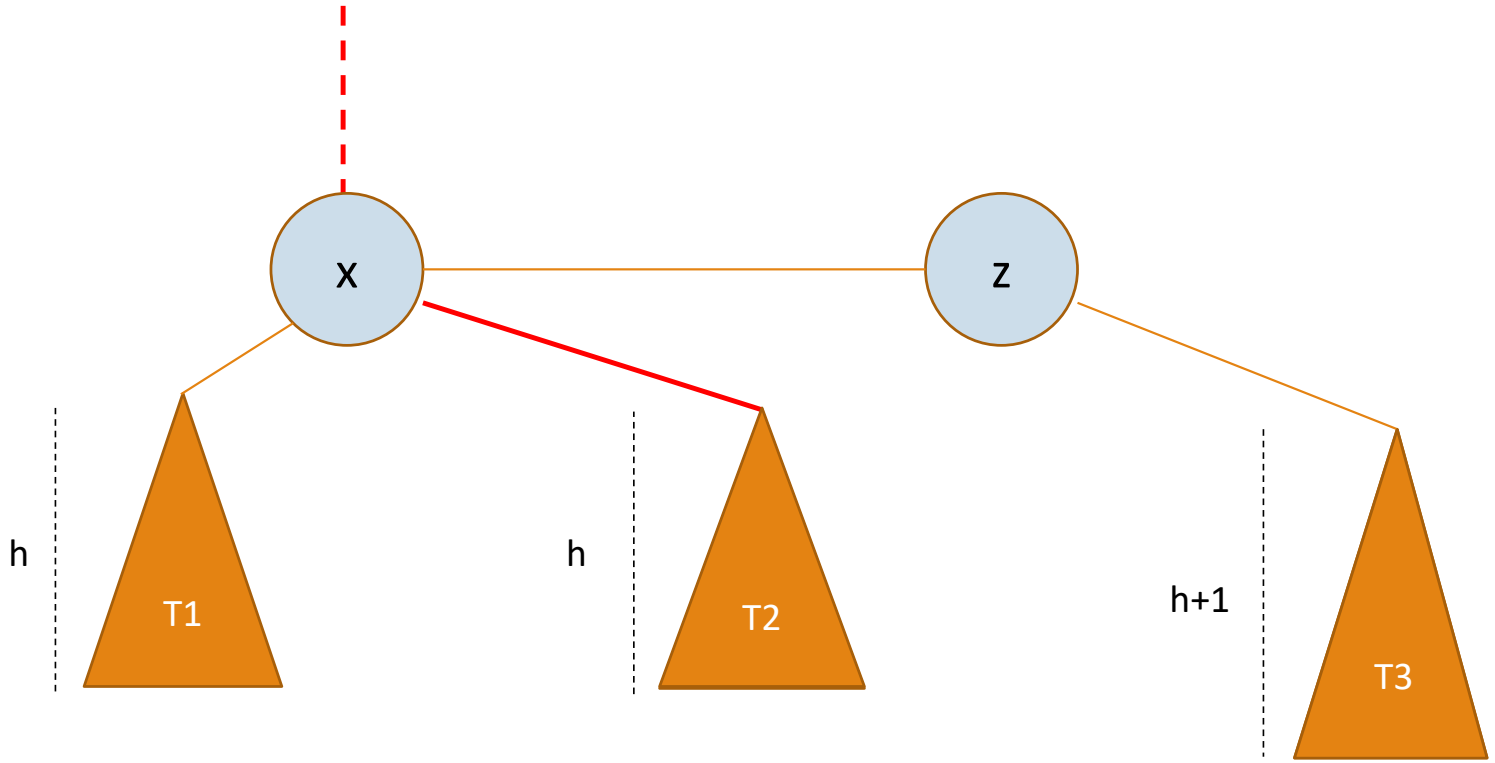
Perform Left Rotation on x!



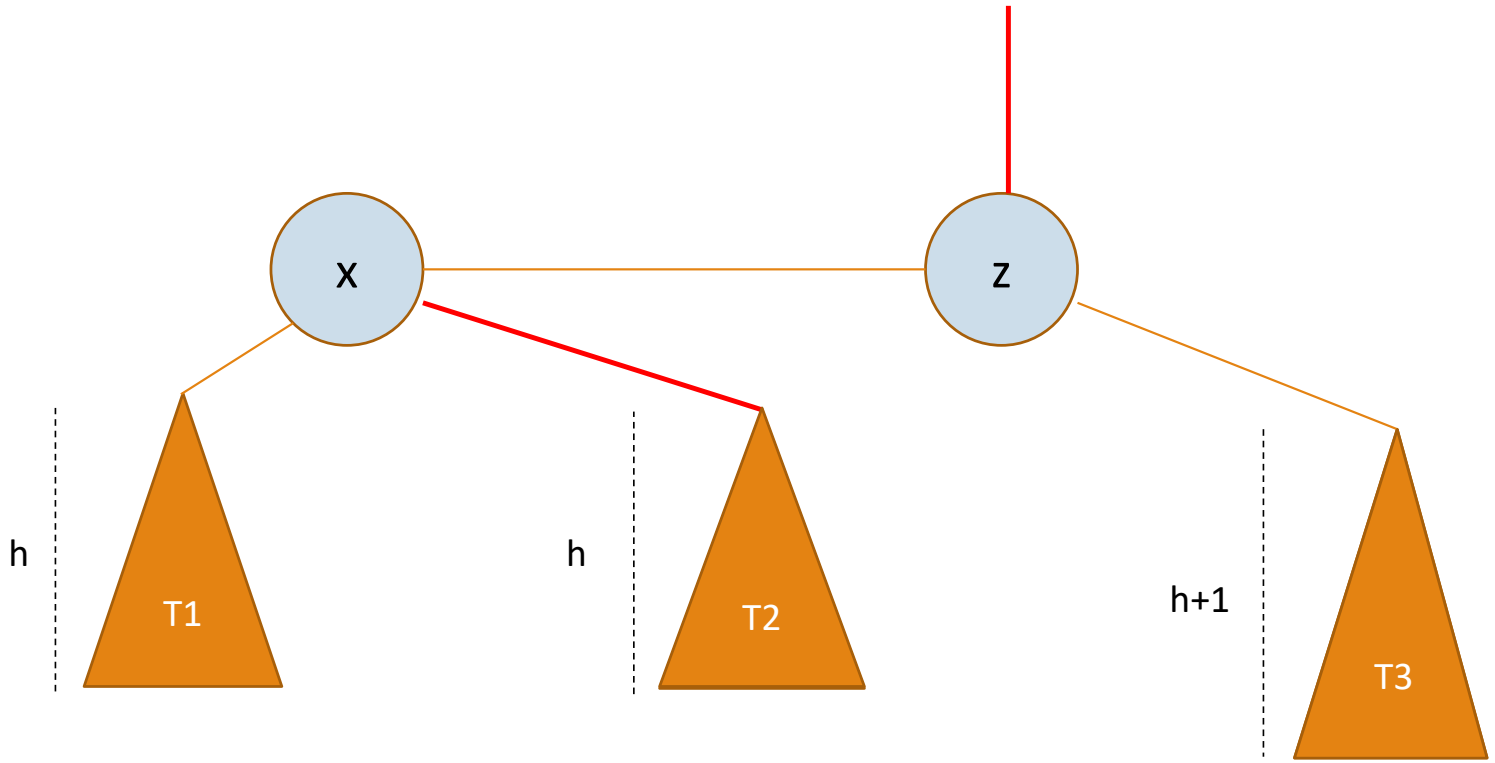
Rotate Left (x)



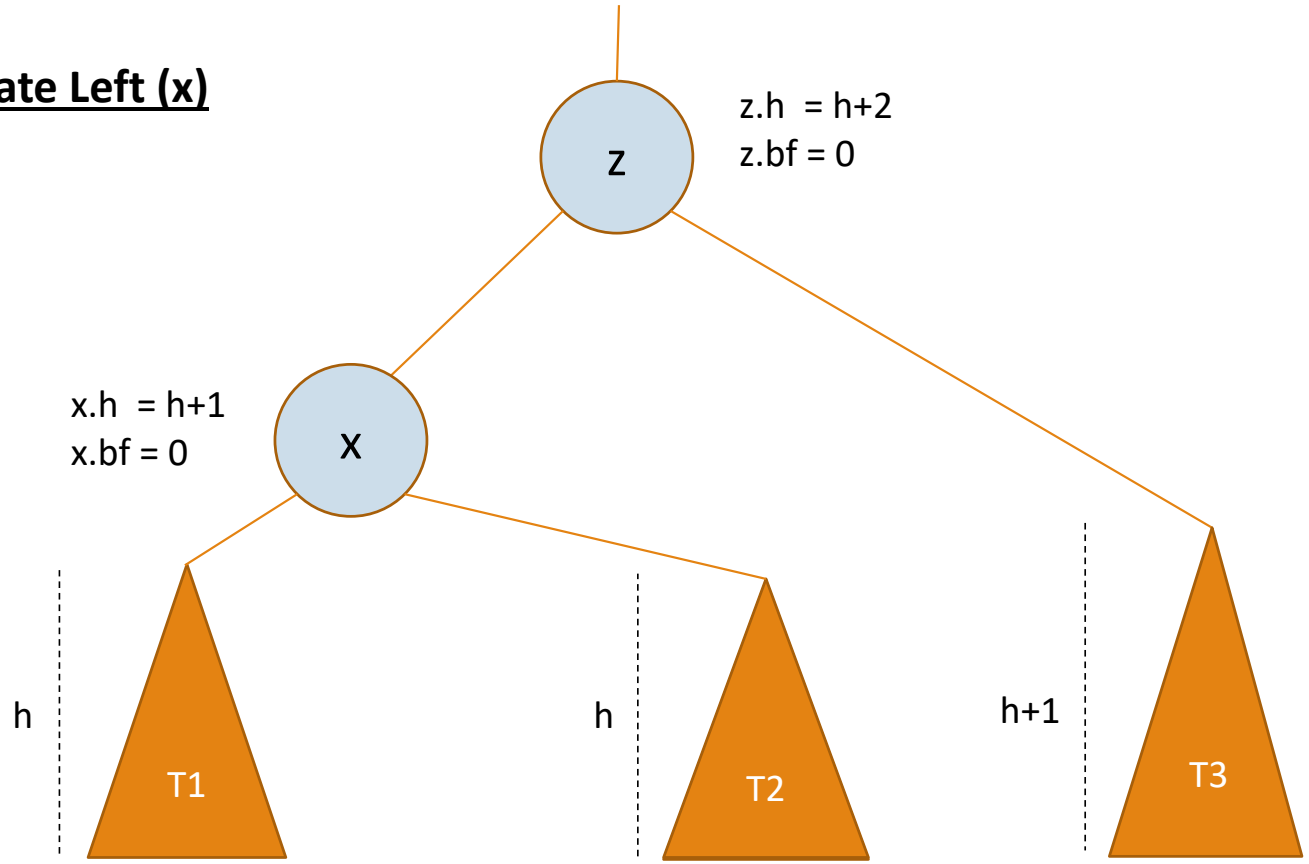
Rotate Left (x)



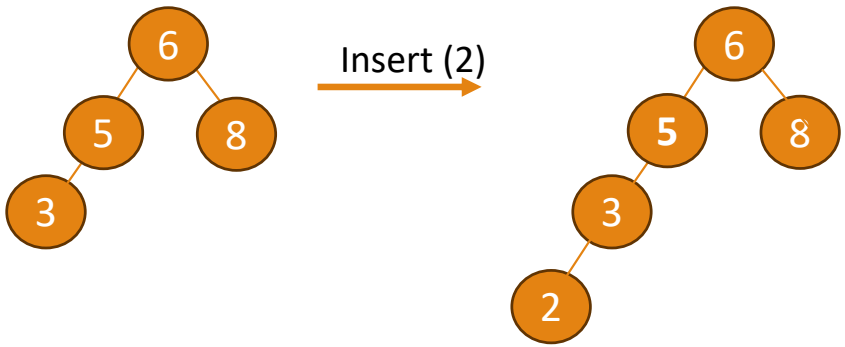
Rotate Left (x)



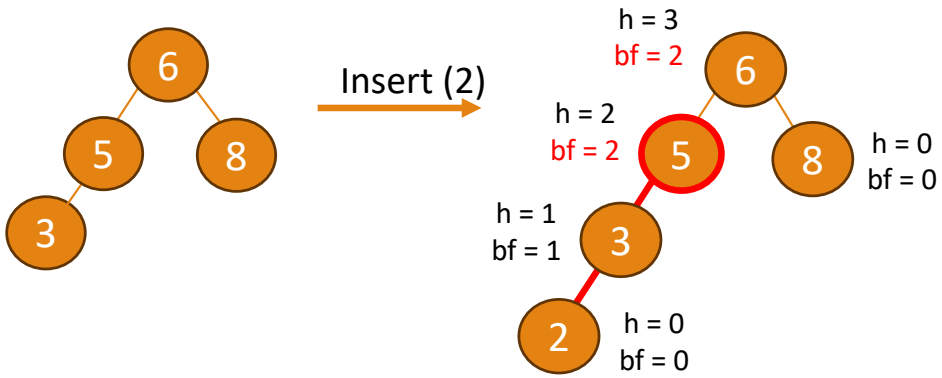
Rotate Left (x)



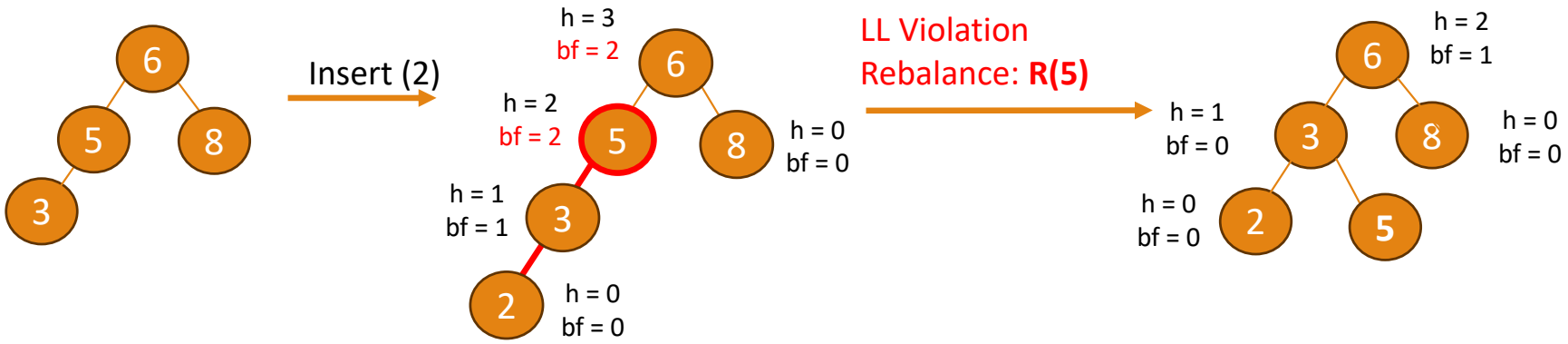
Insertion - Example



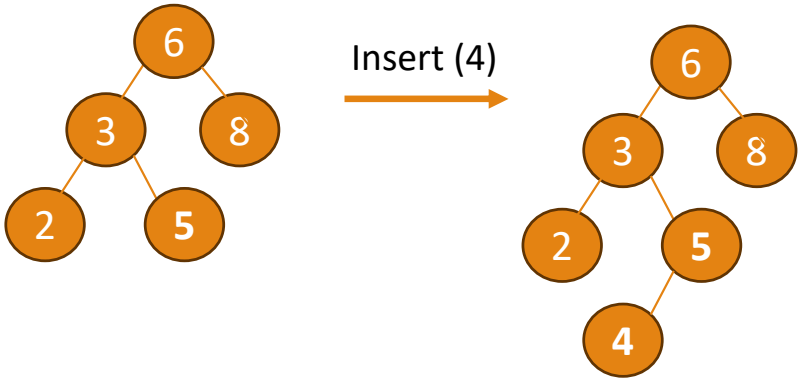
Insertion - Example



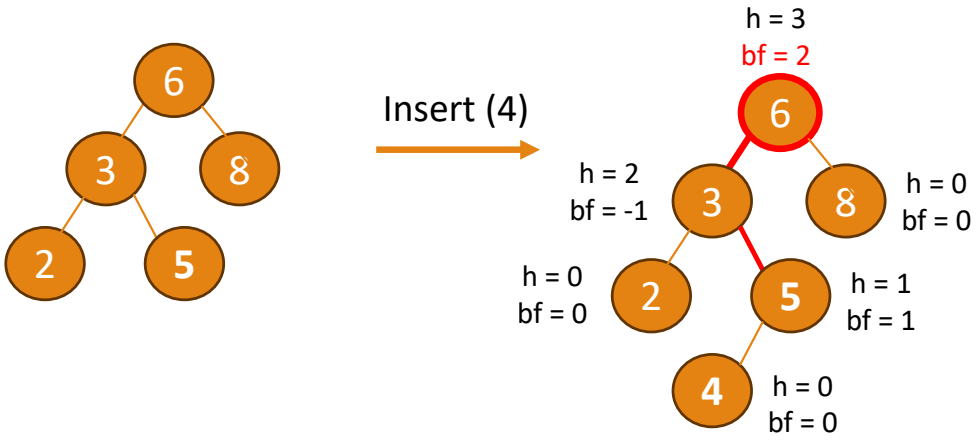
Insertion - Example



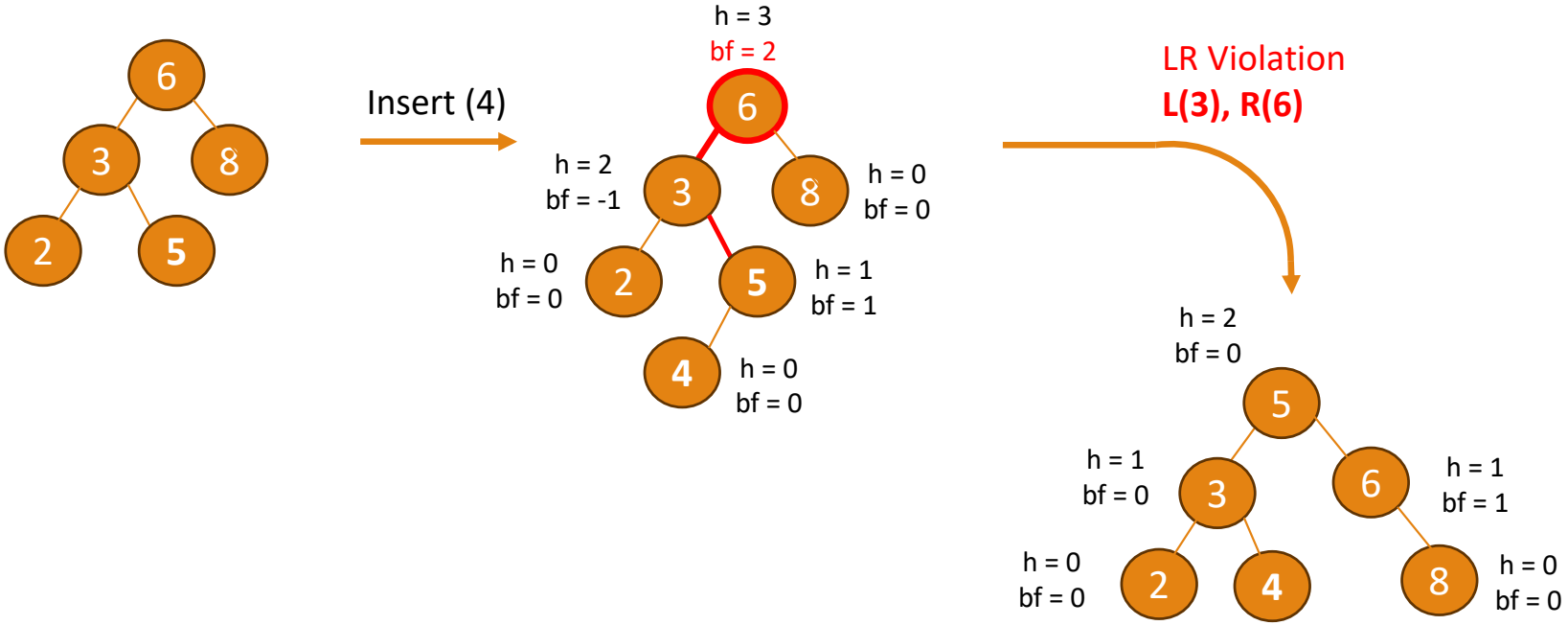
Insertion - Example



Insertion - Example



Insertion - Example

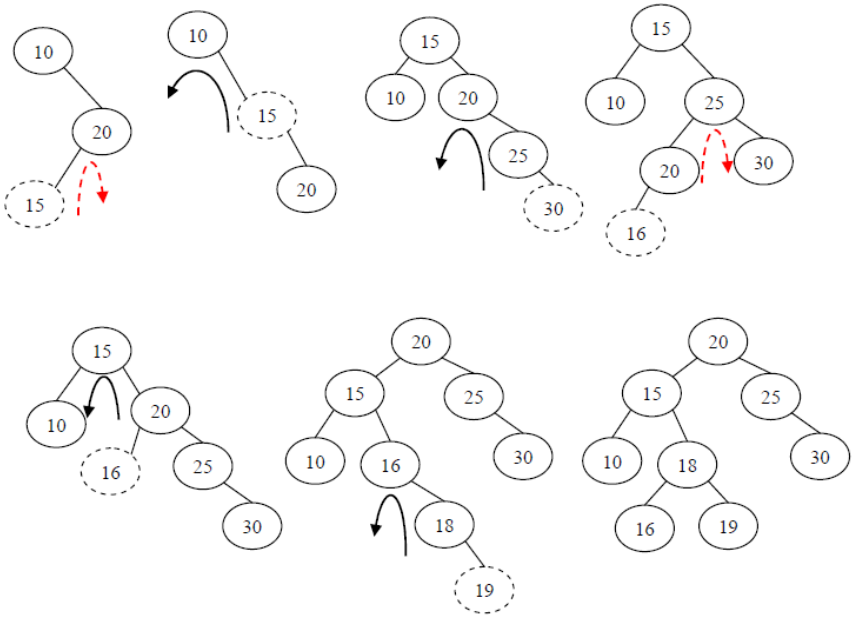


Question 1

- Starting with an empty tree, insert the following elements into an AVL tree:
- 10, 20, 15, 25, 30, 16, 18, 19.

Question 1 - Solution

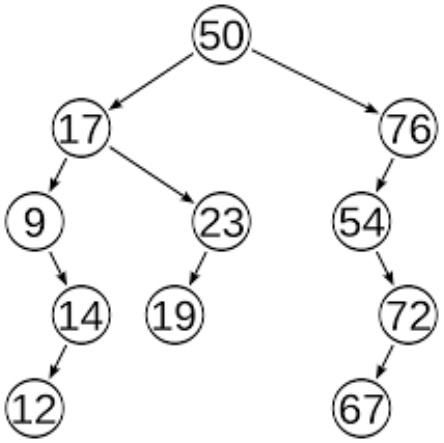
- Starting with an empty tree, insert the following elements into an AVL tree:
- 10, 20, 15, 25, 30, 16, 18, 19.



Question 2

- A node in a binary tree is an **only-child** if it has a parent node but no sibling node
- (Note: The root does not qualify as an only child).
- Prove that in an AVL tree, the number of only-child nodes is at most $\frac{n}{2}$.

Who is **only-child** here?
(not a valid AVL tree)



Question 2

- Claim: if p is an only-child in an AVL tree, then p is a leaf.
- **Proof:**

  Let $\boldsymbol{p}$ be an only-child in an AVL tree.

  Without loss of generality assume p is a left child of some node $\boldsymbol{x}$.

  x does not have a right child because $\boldsymbol{p}$ is an only-child.

  If $\boldsymbol{p}$ has any children, then the balance factor of $\boldsymbol{x}$ is $> \mathbf{1}$, therefore, $\boldsymbol{p}$ is a leaf.

- Thus, in an AVL tree, only leaves may be only-child nodes. Since distinct only children have distinct parents, this implies that for every only-child, there is a unique parent node that is not an **only-child**. Therefore, at most half of the nodes are only children.

Question 3 – Exam-style

An investment manager in a bank needs to keep track of her portfolio.

Each investment has a **name**, a value **v** (in Dollars), and a positive number **r** which is an estimate of the risk associated with the investment (the higher the number **r**, the higher the risk).

You may assume all names, values and risk estimates are distinct.

Describe a data structure supporting the following operations as efficiently as you can:

- Insert(x) - insert a new investment x (x is a structure that includes three fields: **name, v, r**)
- ReportRisky(r) – report the most valuable investment whose risk is more than **r**.
- TotalValueSafe(r) - returns the total value of all current investments whose risk is less than **r**

Question 3 – Solution

We will maintain a balanced BST (say, an AVL tree) in which every node corresponds to an investment and the **key of an investment is its risk**.

Maintain two augmented fields at each node:
- x.sumValue stores the sum of the values of the investments in the subtree rooted at x.

- x.maxValue stores the maximum value of an investment in the subtree rooted at x.

The augmented fields at node x can be computed from x.v and the fields of x.left and x.right in O(1) time. Hence this augmentation can be maintained during the standard BST operations in O(log n) time.

Insert(x) - as in a regular BST, taking O(log n) time.

TotalValueSafe(r) - The same as in "Sum of Keys < x" example from Lecture 8.

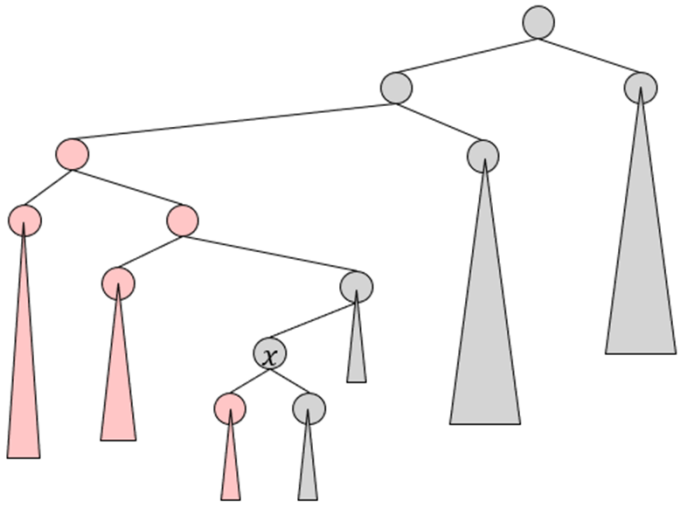
ReportRisky(r) – Similar idea (Pseudo-Code ahead)

Question 3 Solution - TotalValueSafe(r)

TotalValueSafe(r) - returns the total value of all current investments whose risk is less than **r.**

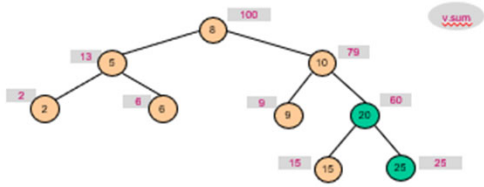
Augmented data: Subtree Sum

Where are the elements smaller than x



SumSmaller

SumSmaller (x,k)
if x==null, then return 0
if k == *x.key*
  then $s \leftarrow$ *(x.left).sum*
elseif $k <$ *x.key*
  then $s \leftarrow$ *SumSmaller*(x.left,k)
else $s \leftarrow$ *x.key*+ *(x.left).sum* +*SumSmaller*(x.right,k)
return s



Initial call: *SumSmaller*(T,k)

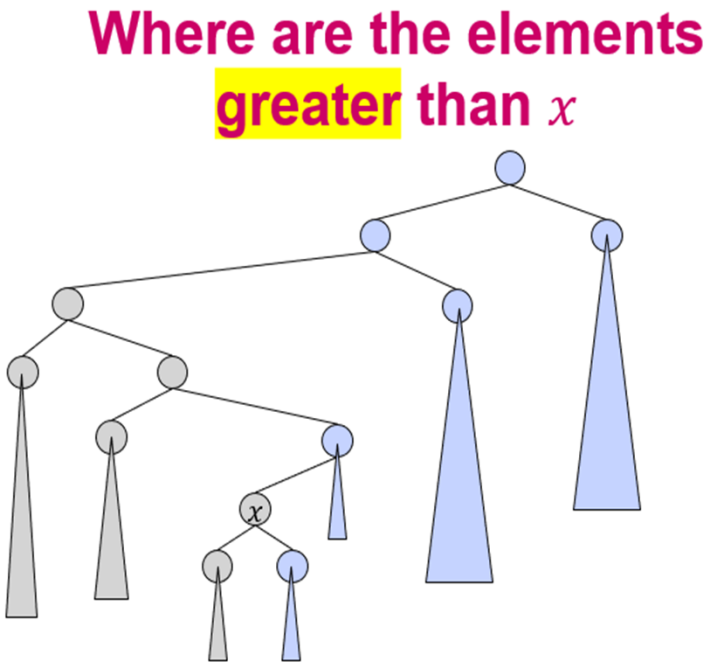
Worst case running time: Θ(h)

T: root of the tree
x: a node
k: a key

Question 3 Solution - ReportRisky(r)

ReportRisky(r) – report the most valuable investment whose risk is more than **r**.

Augmented data: Max Value in Subtree.



27

ReportRiskyHelper(r,x):

```
if x == null: return 0

if x.r ≤ r
        return ReportRiskyHelper(r, x.right)

if x.r > r

        then cur_max = ReportRiskyHelper(r, x.left)

if (x.right ≠ null and x.right.max_value > cur_max)

        then cur_max = x.right.max_value

if x.v > cur_max return x.v

        else return cur_max
```

Question 4.1

Suggest a linear-time algorithm that augments a BST to maintain a height field at each node.

Question 4.1

Suggest a linear-time algorithm that augments a BST to maintain a height field at each node.

Solution: Traverse the tree in post-order and preform the following modifications:

- Base Case: If a node v has no children, set v.height = 0.
- Otherwise: v.height = 1 + the maximal height among its children.

Question 4.2

Suggest a linear-time algorithm that checks whether a BST with a height field is an AVL tree.

Question 4.2

Suggest a linear-time algorithm that checks whether a BST with a height field is an AVL tree.

Solution: Modify the last solution to verify the correctness of all height fields in addition to the balance factor at each node.

(Pseudo code in the following slide.)

CHECK-AVL(node)

```
if node == NIL
    return (TRUE, -1)  // Pair of AVL status and height

(leftIsAVL, leftHeight) = CHECK-AVL(node.left)

if leftIsAVL == FALSE
    return (FALSE, 0)

(rightIsAVL, rightHeight) = CHECK-AVL(node.right)

if rightIsAVL == FALSE
    return (FALSE, 0)

isAVL = ABS(leftHeight - rightHeight) <= 1
if node.height != MAX(leftHeight, rightHeight) + 1
    return (FALSE, 0)

return (isAVL, node.height)
```

Question 5.1 – true/false

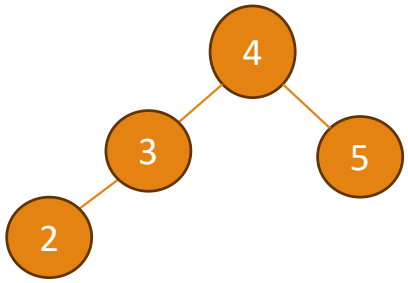
In an AVL tree, all leaves have the same depth.

Question 5.1 – true/false

In an AVL tree, all leaves have the same depth.

False

Consider the following tree:



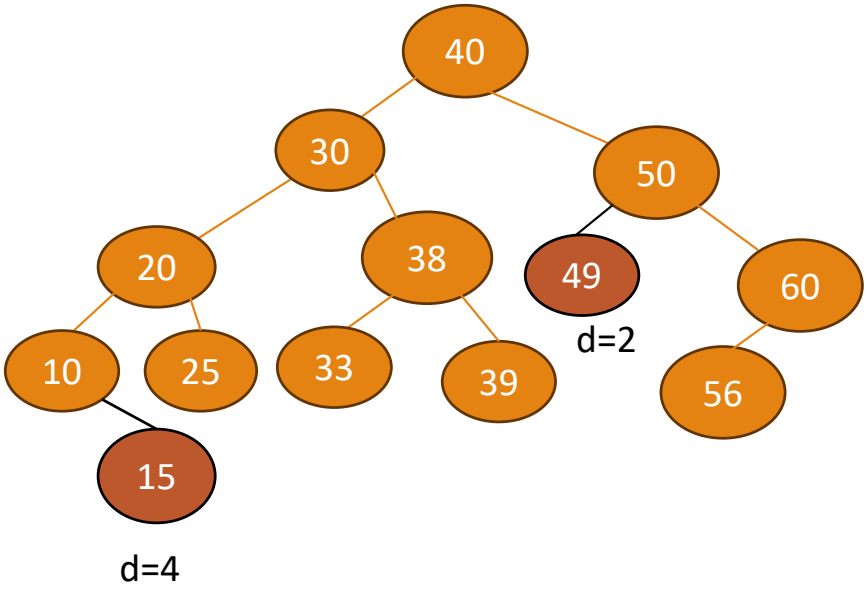
Question 5.2 – true/false

In an AVL tree, the maximal gap between the depths of two leaves is at most 1.

Question 5.2 – true/false

False

Consider the following tree:



Question 5.3 – true/false

For any integer x, there is an AVL tree in which there are two leaves u, v, such that the gap between the depths of u and v is at least x.

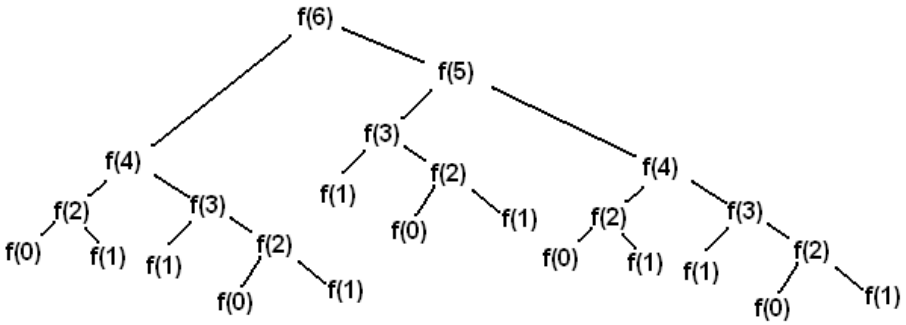
Question 5.3 – true/false

Proof: a Fibonacci tree.

A Fibonacci tree of order k, f(k), is defined recursively:

f(0) and f(1) are simply a single node.

for k > 1, the root of f(k) has a left subtree f(k-2), and a right subtree f(k-1).



Example: A Fibonacci tree of order 6

Question 5.3 - Fibonacci tree

It is easy to verify that a Fibonacci tree is an AVL tree.

Note: the tree represents the recursive calls performed when calculating Fib(k).

The leftmost leaf has depth $\left\lfloor \frac{k}{2} \right\rfloor$.

The rightmost leaf has a depth of k-1.

Therefore, for any given x, in f(2x+1) the gap is x